

LEVEL 1 PROJECT REPORT

Static Website Deployment on AWS S3 with Amazon CloudFront

Static HTML/CSS/JavaScript Portfolio Hosting with CloudFront CDN and Origin Access Control

AWS Region: Asia Pacific (Mumbai) — ap-south-1

Technology / Service	Role in Project
Amazon S3	Stores and serves the static website as the CloudFront origin
Amazon CloudFront	CDN layer for HTTPS delivery and caching
Origin Access Control (OAC)	Restricts direct origin access and allows CloudFront to access S3
AWS CLI	Uploads/updates website files from the local machine
HTML / CSS / JavaScript	Static portfolio website source
GitHub	Source-code/version-control stage of the Level 1 workflow

Prepared by Manimaran

Implementation evidence: 23 screenshots + 1 system architecture diagram

Table of Contents

1. Project Overview
 2. Problem Statement
 3. Objectives
 4. Technologies and Tools
 5. Prerequisites
 6. System Architecture
 7. Deployment Workflow
 8. Project Components
 9. Implementation Procedure
 10. Testing and Validation
 11. Troubleshooting and Operational Notes
 12. Security Considerations
 13. Advantages
 14. Limitations
 15. Future Enhancements
 16. Key Lessons Learned
 17. Interview / Viva Explanation
 18. Conclusion
- Appendix A — Implementation Evidence Inventory

1. Project Overview

This Level 1 project demonstrates the deployment of a static portfolio website using Amazon S3 and Amazon CloudFront. The website is built from HTML, CSS, and JavaScript files, stored in an S3 bucket named “mpg-portfolio”, and delivered to users through a CloudFront distribution. The implementation also uses Origin Access Control (OAC) so that CloudFront can access the S3 origin while direct origin access can be restricted.

The implementation evidence shows S3 bucket creation and object management, CloudFront distribution creation and origin configuration, OAC-related settings and bucket policy, CloudFront distribution details, browser validation, AWS CLI installation/configuration, CLI-based S3 inspection, website-file updates, and final CloudFront delivery.

2. Problem Statement

A static website needs a reliable way to store web assets and make them accessible to users. The project addresses this by using Amazon S3 for object storage and Amazon CloudFront for content delivery. The implementation also demonstrates how a CloudFront distribution can be connected to an S3 origin and how access control can be applied between the CDN and the bucket.

3. Objectives

- Create an Amazon S3 bucket for static website content.
- Upload HTML, CSS, and JavaScript website files to S3.
- Configure Amazon CloudFront with the S3 bucket as the origin.
- Configure Origin Access Control (OAC) for CloudFront-to-S3 access.
- Apply the required S3 bucket policy for CloudFront access.
- Access the deployed portfolio through the CloudFront distribution domain.
- Validate HTTPS-based delivery through CloudFront.
- Use AWS CLI to inspect and update the S3-hosted website.
- Understand caching and the need for CloudFront invalidation when content changes.
- Maintain the project source code and documentation in GitHub as part of the Level 1 workflow.

4. Technologies and Tools

Technology / Service	Purpose	Role in Project
Amazon S3	Object storage	Stores website files
Amazon CloudFront	Content delivery network	Delivers cached website content to users
Origin Access Control	Origin security	Allows CloudFront to securely access the S3 origin
AWS CLI	Command-line AWS management	Lists buckets/objects and supports website updates
HTML	Web structure	Defines portfolio content
CSS	Presentation	Defines website layout and styling
JavaScript	Client-side behavior	Adds interactive behavior
GitHub	Source control	Stores project source and README as part of the workflow
Web Browser	Validation	Tests the final public website

5. Prerequisites

- AWS account with access to Amazon S3 and Amazon CloudFront.
- A static website containing HTML, CSS, JavaScript, and related assets.
- An AWS S3 bucket created in the Mumbai region (ap-south-1) for this implementation.
- AWS CLI installed and available from Windows Command Prompt.
- AWS CLI configured with an appropriate IAM identity.
- A modern web browser for validation.
- A GitHub repository for the source code and project README.

6. System Architecture

The Level 1 architecture is a simple serverless/static-content delivery path. Website source files are prepared on the user's computer and uploaded to Amazon S3. Amazon CloudFront uses the S3 bucket as its origin. OAC and the corresponding S3 bucket policy control origin access. End users access the CloudFront distribution, which serves the website over HTTPS and can cache static content at CloudFront edge locations.

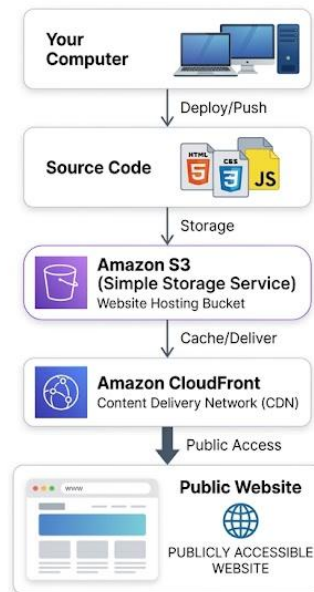


Figure 1. Level 1 system architecture — Computer → Source Code → Amazon S3 → Amazon CloudFront → Public Website.

6.1 Architecture Flow

Step	Flow	Purpose
1	User Computer → Source Code	Website HTML/CSS/JS files are created and maintained locally.
2	Source Code → Amazon S3	Website files are uploaded and stored as S3 objects.
3	Amazon S3 → CloudFront Origin	CloudFront is configured to use the S3 bucket as its origin.
4	CloudFront → User Browser	CloudFront delivers website content through its distribution domain.
5	CloudFront ↔ S3 using OAC	CloudFront is authorized to retrieve content from the protected S3 origin.

7. Deployment Workflow

- 1. Create and verify the S3 bucket.
- 2. Upload the initial website files to the S3 bucket.
- 3. Open Amazon CloudFront and begin creating a distribution.
- 4. Select the S3 bucket as the origin.
- 5. Configure Origin Access Control (OAC).
- 6. Complete the CloudFront distribution configuration.
- 7. Update the S3 bucket policy to permit the CloudFront service principal for the distribution.
- 8. Verify the CloudFront distribution becomes enabled.
- 9. Test the CloudFront domain in a browser.
- 10. Use AWS CLI to verify the CLI installation and S3 access.
- 11. Inspect the local website directory and update S3 content using CLI commands.
- 12. Re-test the website through CloudFront and account for caching when files change.
- 13. Maintain the source and README in GitHub as part of the project workflow.

8. Project Components

Component	Description
S3 Bucket — mpg-portfolio	Primary storage location for the static portfolio website.
Website Objects	HTML/CSS/JS and related static assets uploaded into S3.
CloudFront Distribution	Public delivery layer for the website.
CloudFront Origin	The S3 bucket selected as the content origin.
Origin Access Control	Security mechanism used to authorize CloudFront access to S3.
S3 Bucket Policy	Policy that grants the CloudFront service principal access required to retrieve objects.
AWS CLI	Command-line interface used to verify configuration and perform S3 operations.
Browser	Used to validate direct/public website behavior and final CloudFront delivery.
GitHub Repository	Source-control location for website code and README in the Level 1 workflow.

9. Implementation Procedure

The following 23 Screenshots are presented in the chronological order.Each step instruction/action is placed before its corresponding image.

Step 1 — Create/verify the S3 bucket

Instruction / Action: The Amazon S3 console shows the general-purpose bucket list in the Mumbai region, including the project bucket used for the portfolio.

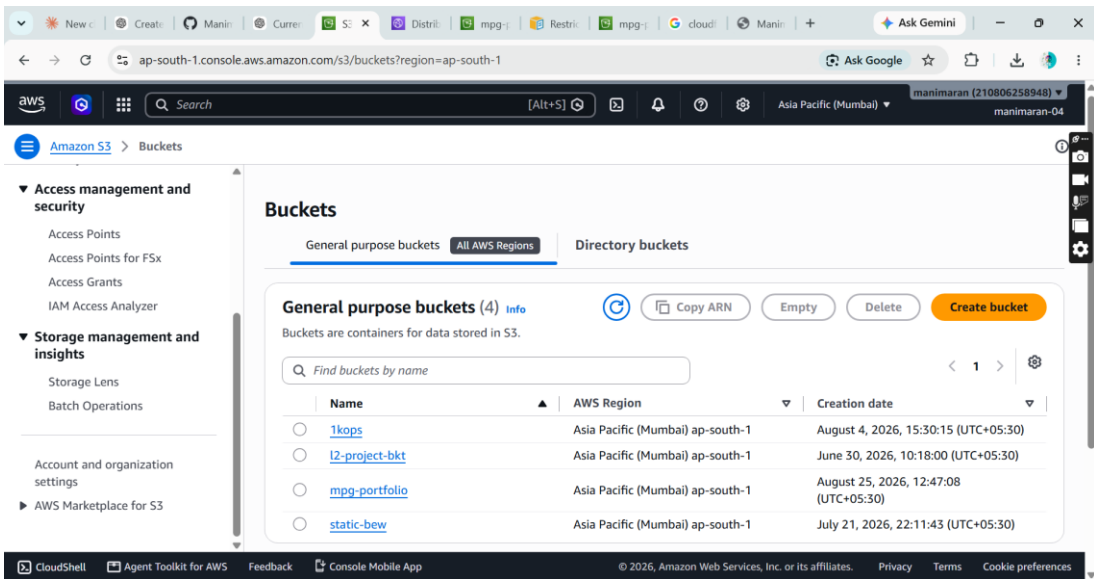


Figure 2. Create/verify the S3 bucket.

Step 2 — Open the mpg-portfolio bucket

Instruction / Action: The mpg-portfolio bucket is opened and the initial website objects are visible in the S3 object list.

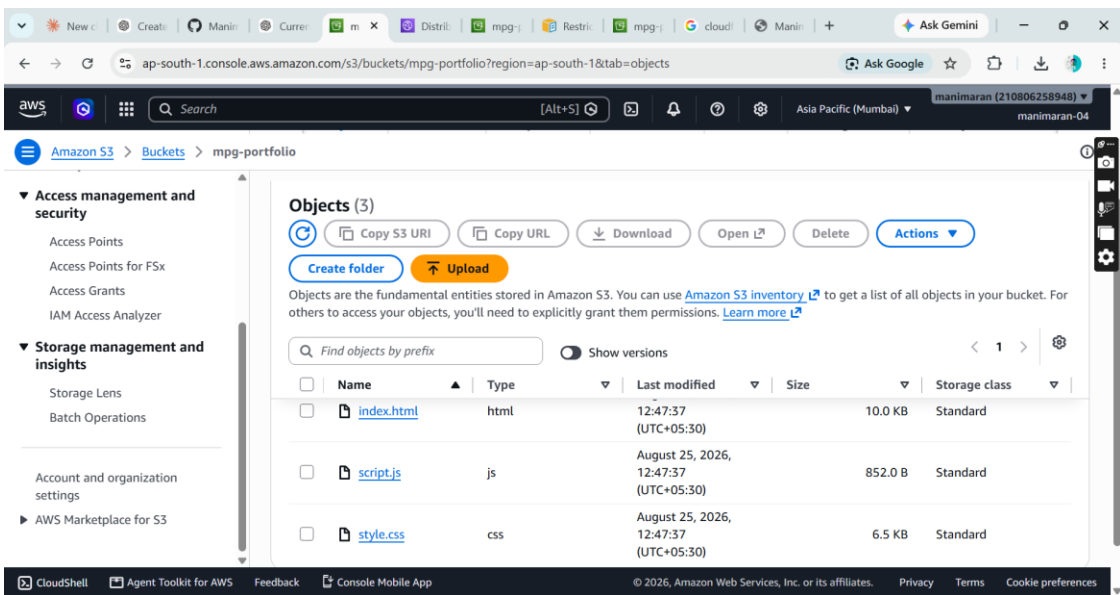


Figure 3. Evidence screenshot — Open the mpg-portfolio bucket.

Step 3 — Check CloudFront distributions before creation

Instruction / Action: The CloudFront Distributions page is opened and initially shows that no distribution exists.

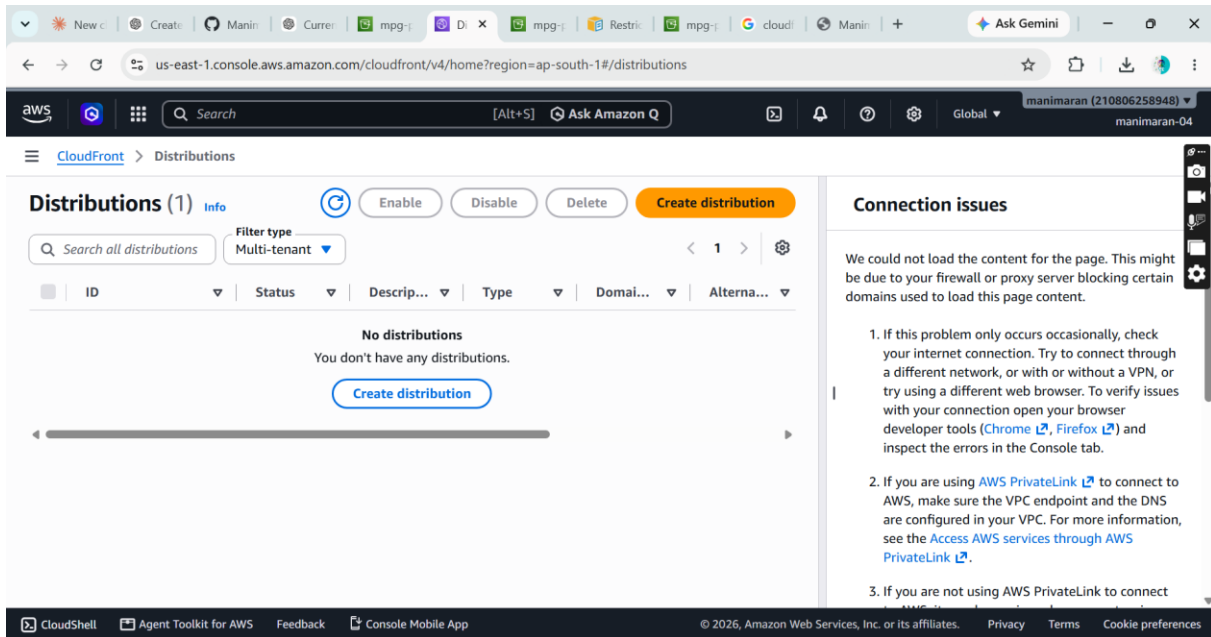


Figure 4. Check CloudFront distributions before creation.

Step 4 — Start CloudFront distribution creation

Instruction / Action: The CloudFront Create Distribution workflow is opened from the Get started screen.

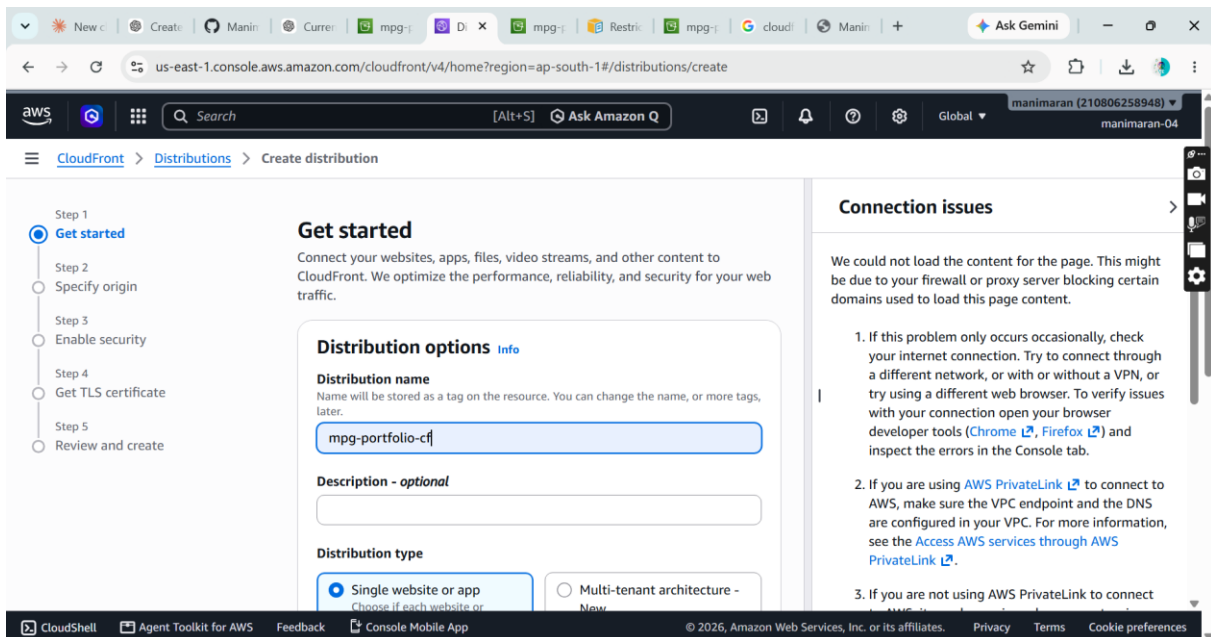


Figure 5. Start CloudFront distribution creation.

Step 5 — Select the S3 origin type

Instruction / Action: The CloudFront creation workflow shows the origin selection stage with an AWS origin/S3 option.

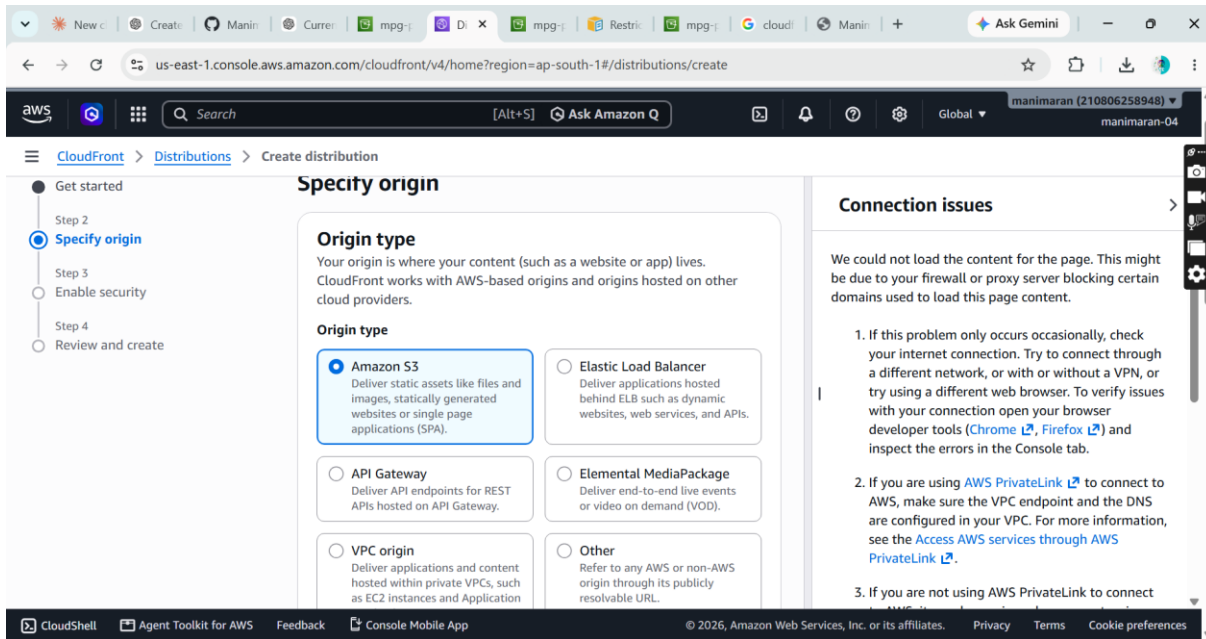


Figure 6. Select the S3 origin type.

Step 6 — Specify the S3 origin

Instruction / Action: The S3 origin domain for mpg-portfolio is entered/selected for the CloudFront distribution.

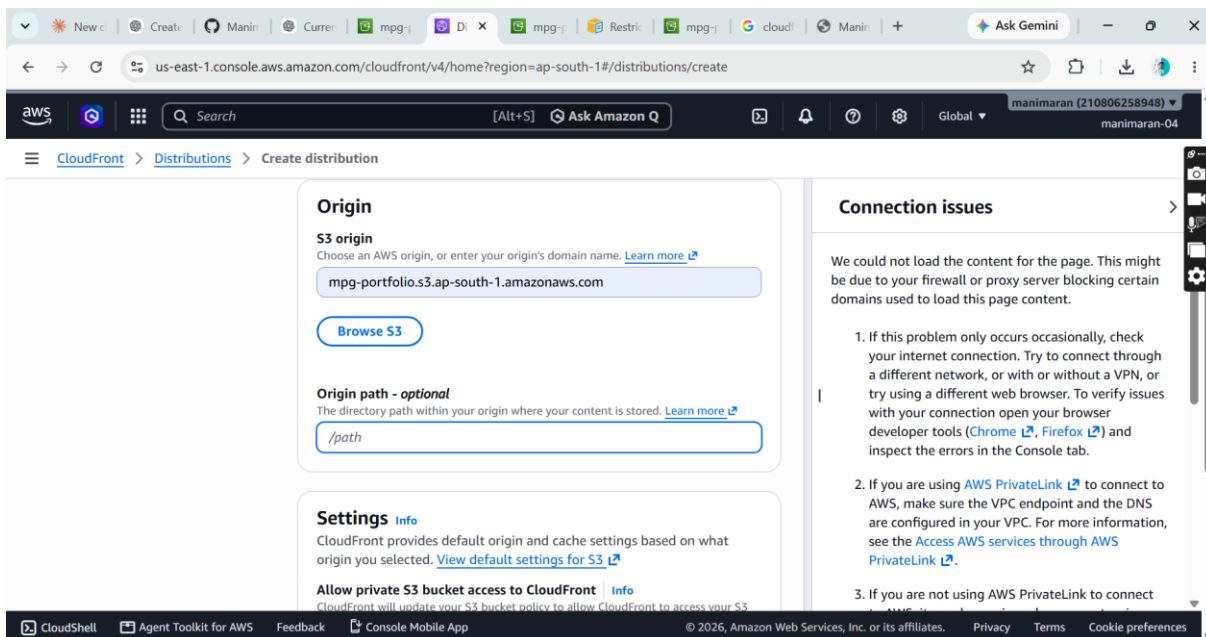


Figure 7. Specify the S3 origin.

Step 7 — Verify the created CloudFront distribution

Instruction / Action: The CloudFront Distributions page shows one enabled standard distribution after creation.

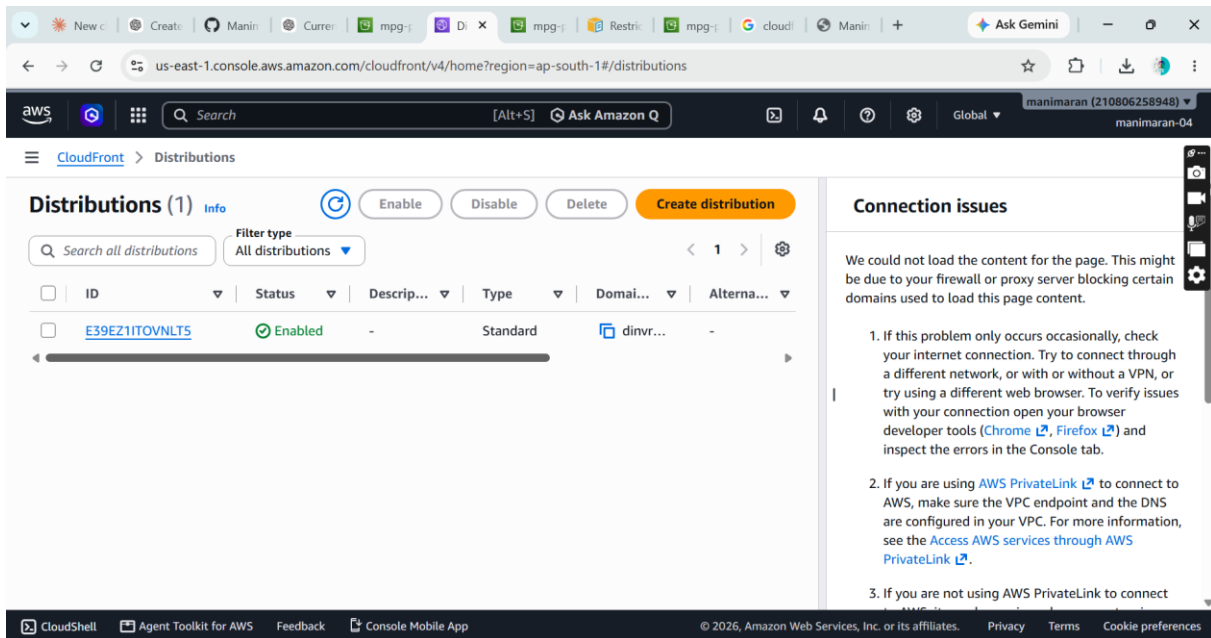


Figure 8. Verify the created CloudFront distribution.

Step 8 — Open the distribution Origins section

Instruction / Action: The distribution details page is opened on the Origins tab to inspect the configured origin.

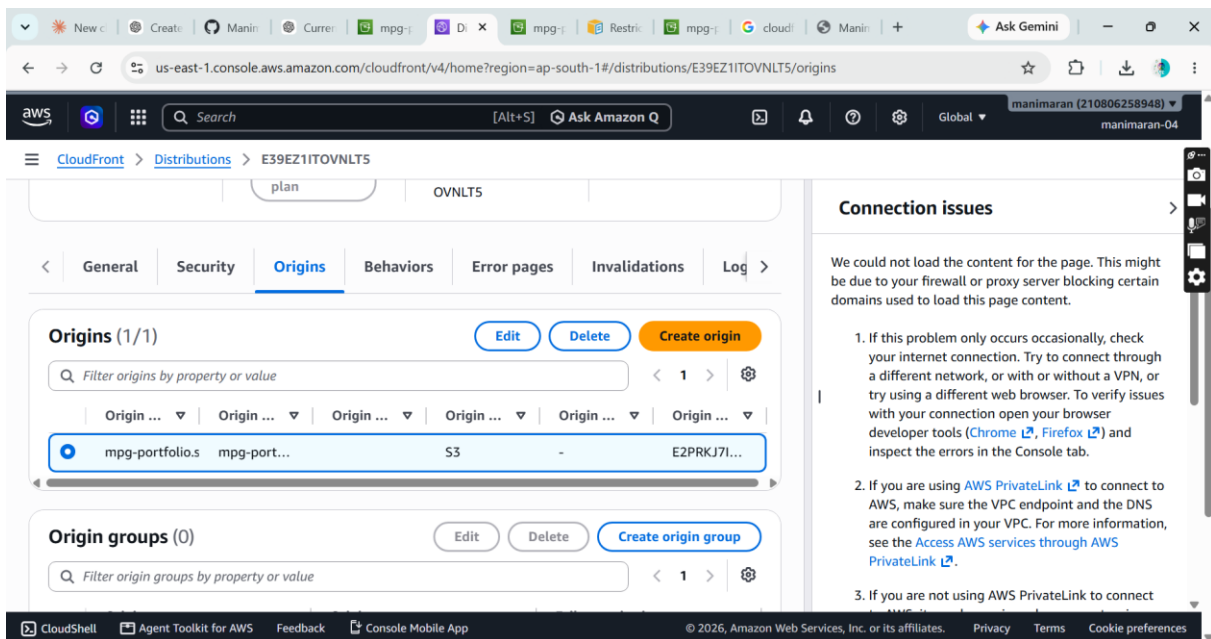


Figure 9. Open the distribution Origins section.

Step 9 — Configure Origin Access Control

Instruction / Action: The origin edit screen shows the recommended Origin Access Control setting, allowing the bucket to restrict access to CloudFront.

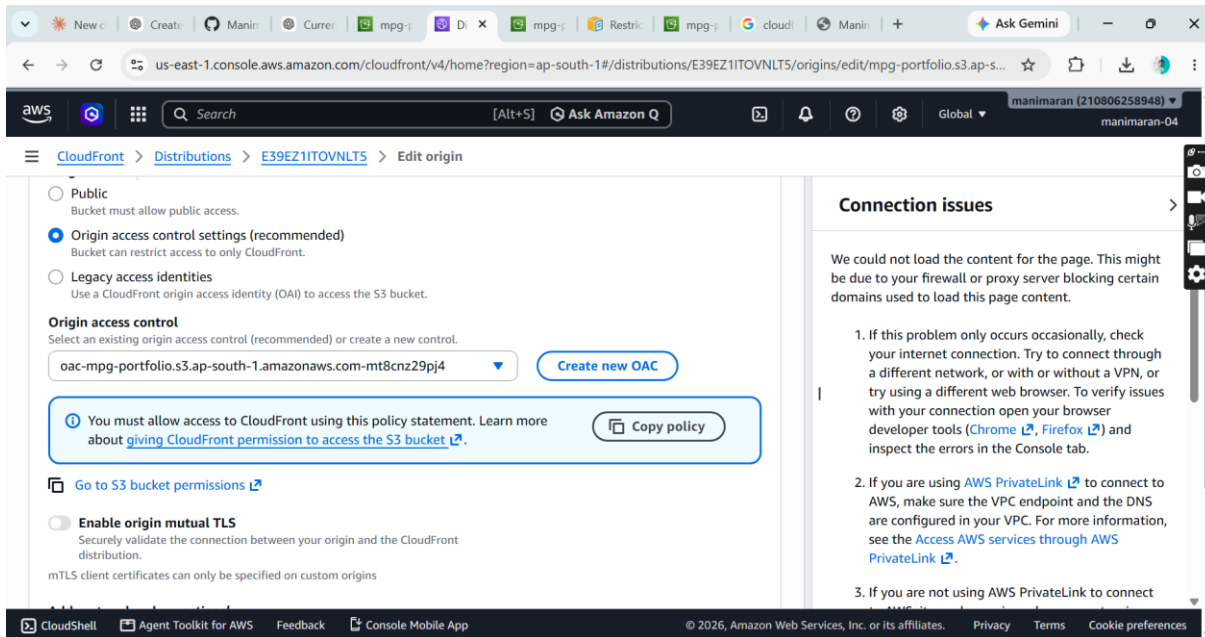


Figure 10. Configure Origin Access Control.

Step 10 — Update the S3 bucket policy

Instruction / Action: The S3 bucket policy editor shows the policy statement used to allow the CloudFront service principal to access the private content.

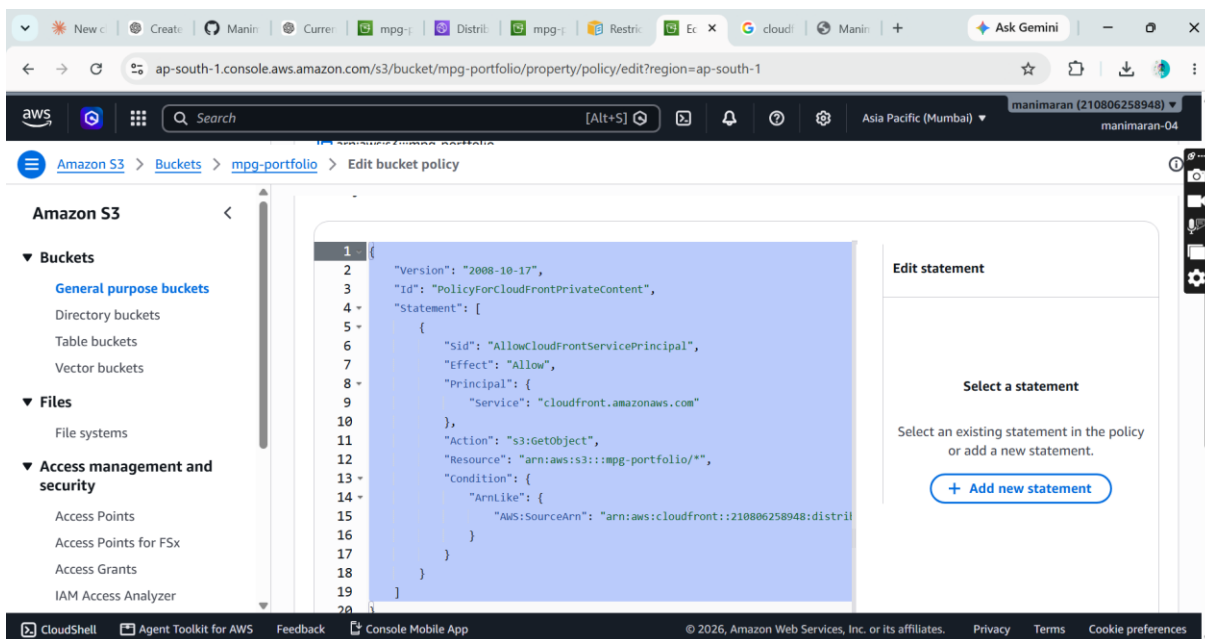


Figure 11. Update the S3 bucket policy.

Step 11 — Review CloudFront distribution details

Instruction / Action: The CloudFront distribution details page shows the project distribution and its CloudFront domain name.

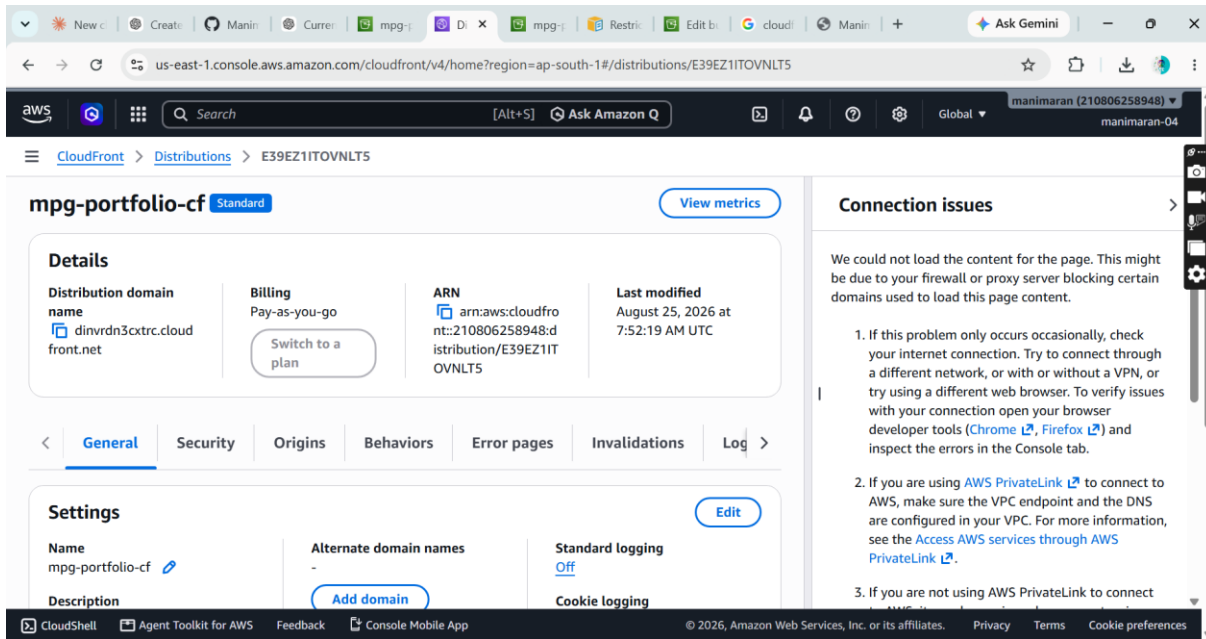


Figure 12. Review CloudFront distribution details.

Step 12 — Validate the public CloudFront website

Instruction / Action: The portfolio website is successfully rendered through the CloudFront distribution domain.

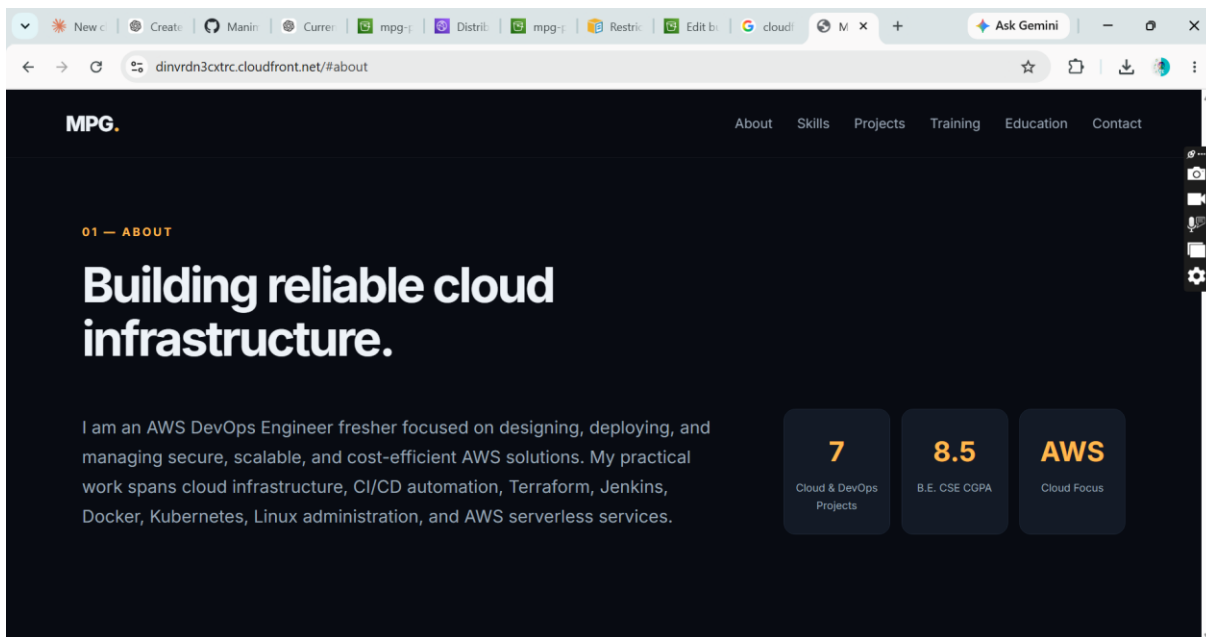


Figure 13. Validate the public CloudFront website.

Step 13 — Verify the local website/source files

Instruction / Action: The local project folder is displayed with the static website content, providing the source files used for deployment.

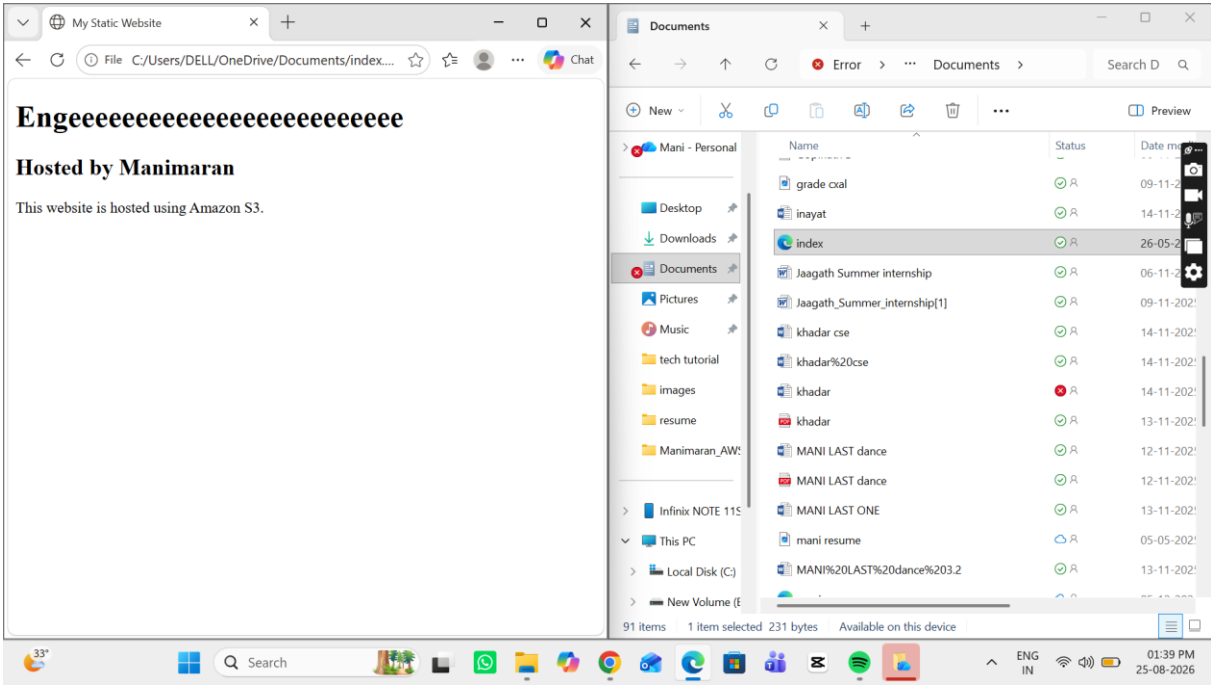


Figure 14. Verify the local website/source files.

Step 14 — Verify uploaded S3 objects

Instruction / Action: The S3 bucket shows the uploaded website objects after deployment/update.

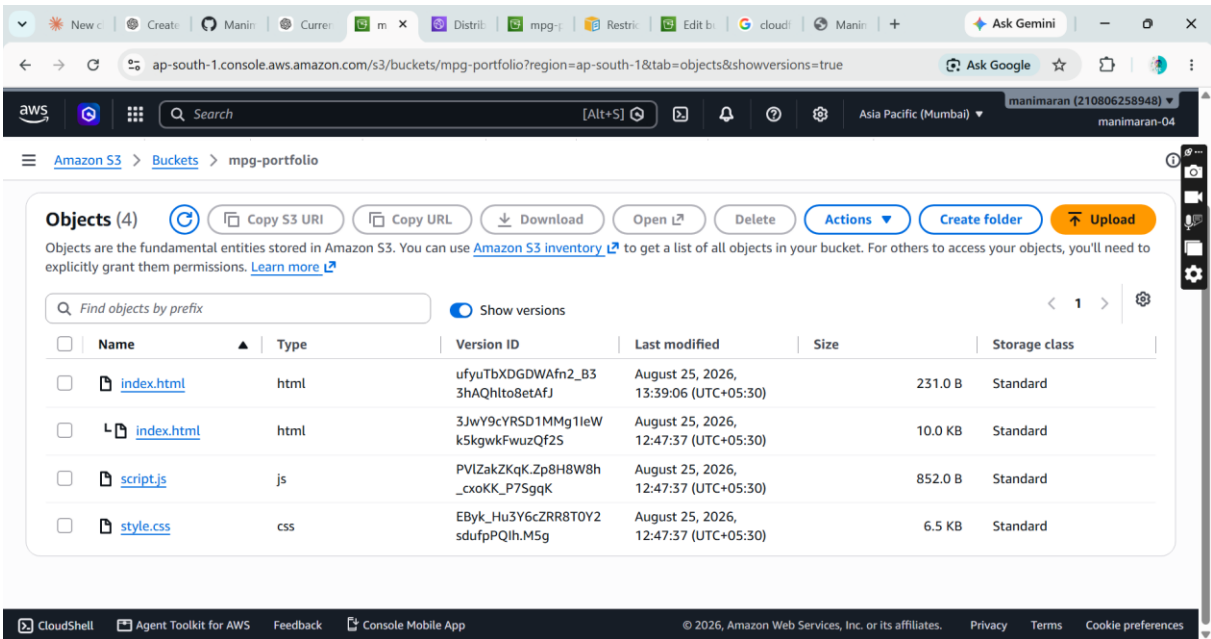


Figure 15. Verify uploaded S3 objects.

Step 15 — Re-test the portfolio through CloudFront

Instruction / Action: The browser again displays the portfolio website through the CloudFront distribution.

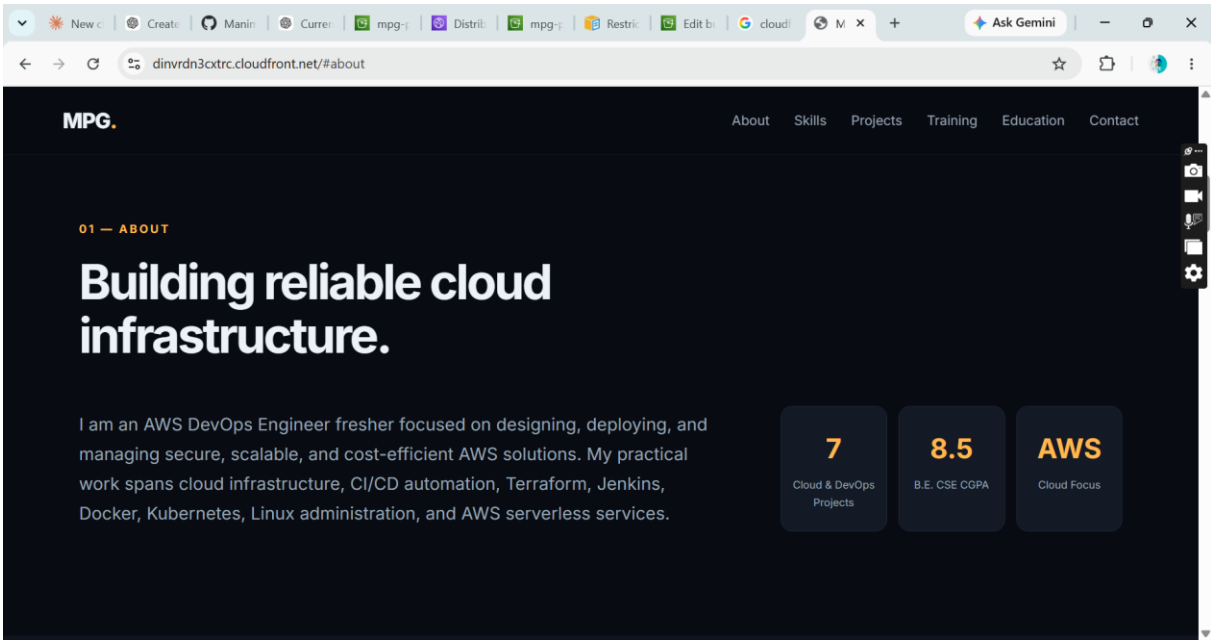


Figure 16. Re-test the portfolio through CloudFront.

Step 16 — Validate direct S3 object access behavior

Instruction / Action: The browser view shows an S3 object URL for index.html, providing evidence of the stored website object and its access behavior.

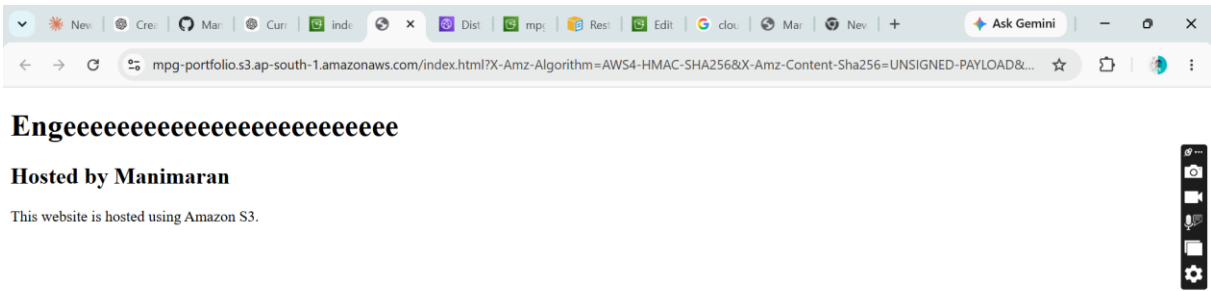
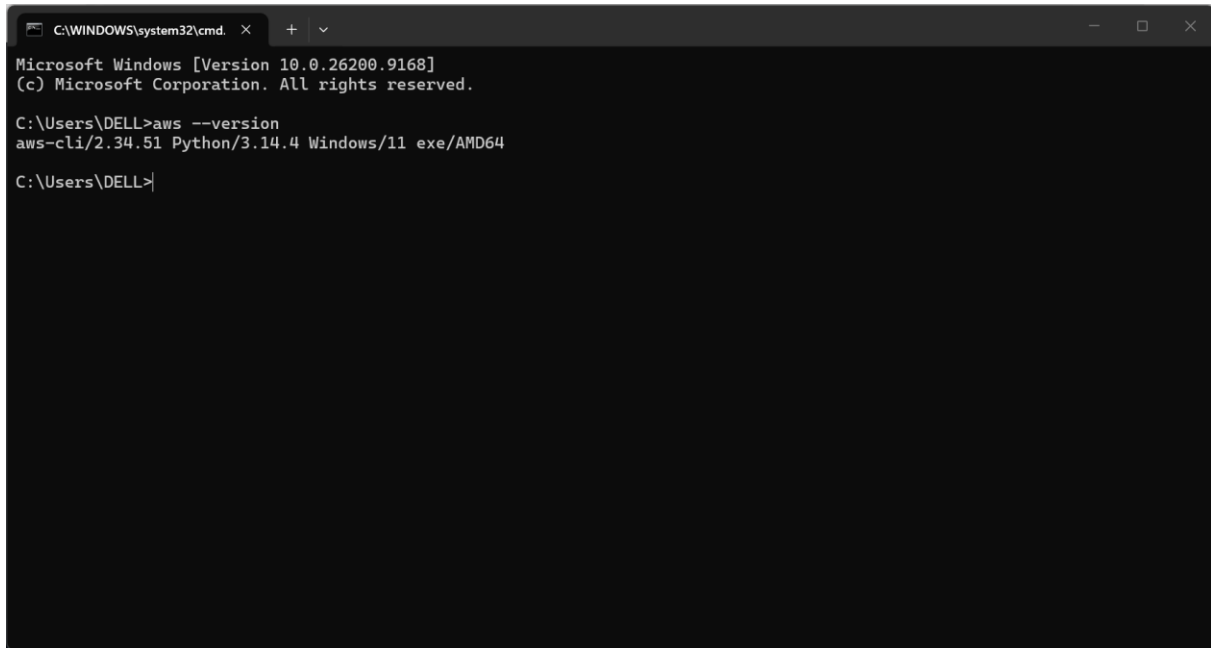


Figure 17. Validate direct S3 object access behavior.

Step 17 — Verify AWS CLI installation

Instruction / Action: Windows Command Prompt shows `aws --version` and confirms that AWS CLI is installed.



```
C:\WINDOWS\system32\cmd. x + v
Microsoft Windows [Version 10.0.26200.9168]
(c) Microsoft Corporation. All rights reserved.

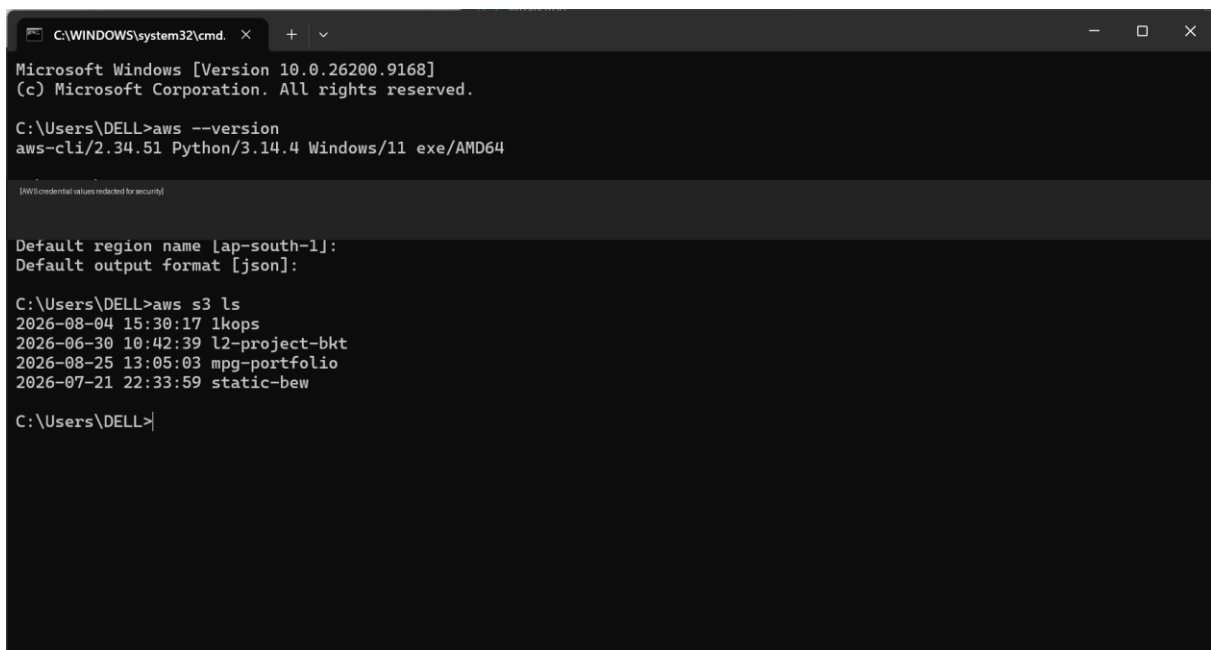
C:\Users\DELL>aws --version
aws-cli/2.34.51 Python/3.14.4 Windows/11 exe/AMD64

C:\Users\DELL>|
```

Figure 18. Verify AWS CLI installation.

Step 18 — Configure AWS CLI

Instruction / Action: The AWS CLI configuration workflow is shown. Credential values have been redacted in this report for security.



```
C:\WINDOWS\system32\cmd. x + v
Microsoft Windows [Version 10.0.26200.9168]
(c) Microsoft Corporation. All rights reserved.

C:\Users\DELL>aws --version
aws-cli/2.34.51 Python/3.14.4 Windows/11 exe/AMD64

[AWS credential values redacted for security]

Default region name [ap-south-1]:
Default output format [json]:

C:\Users\DELL>aws s3 ls
2026-08-04 15:30:17 1kops
2026-06-30 10:42:39 l2-project-bkt
2026-08-25 13:05:03 mpg-portfolio
2026-07-21 22:33:59 static-bew

C:\Users\DELL>|
```

Figure 19. Configure AWS CLI.

Step 19 — Verify S3 access from the CLI

Instruction / Action: The AWS CLI lists available S3 buckets, including mpg-portfolio. Credential values have been redacted in this report for security.



Figure 20. Verify S3 access from the CLI.

Step 20 — Inspect the local project directory

Instruction / Action: The Command Prompt is used to navigate into the local Manimaran_AWS_DevOps_Portfolio directory and inspect its contents.

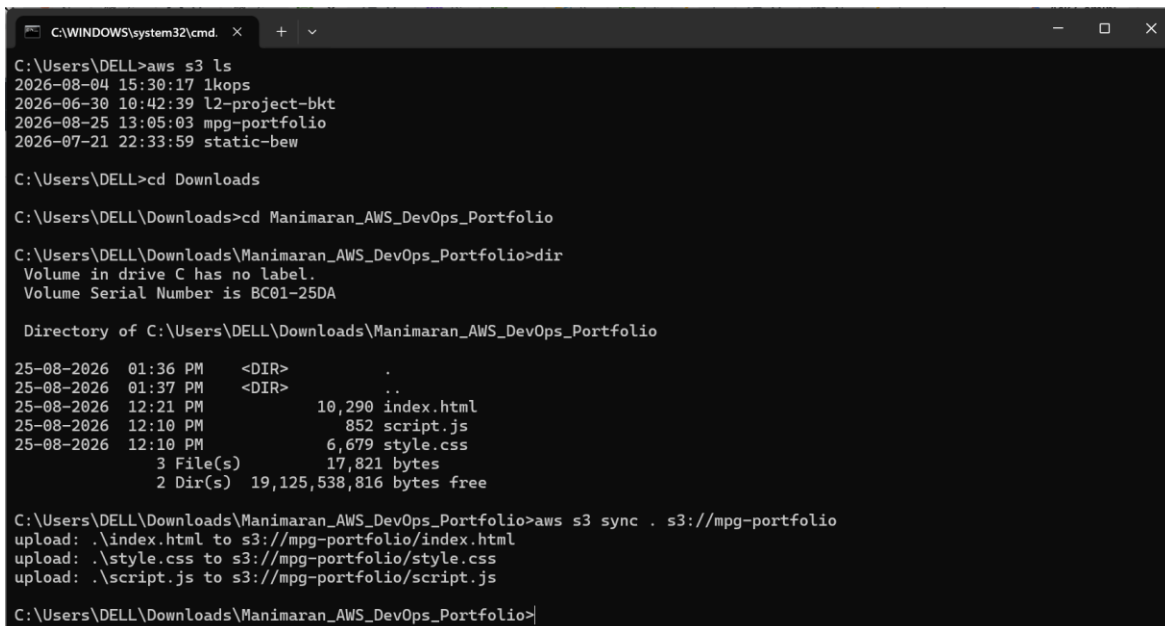


Figure 21. Inspect the local project directory.

Step 21 — Verify the updated S3 bucket contents

Instruction / Action: The S3 console shows the project objects after the CLI/source update workflow.

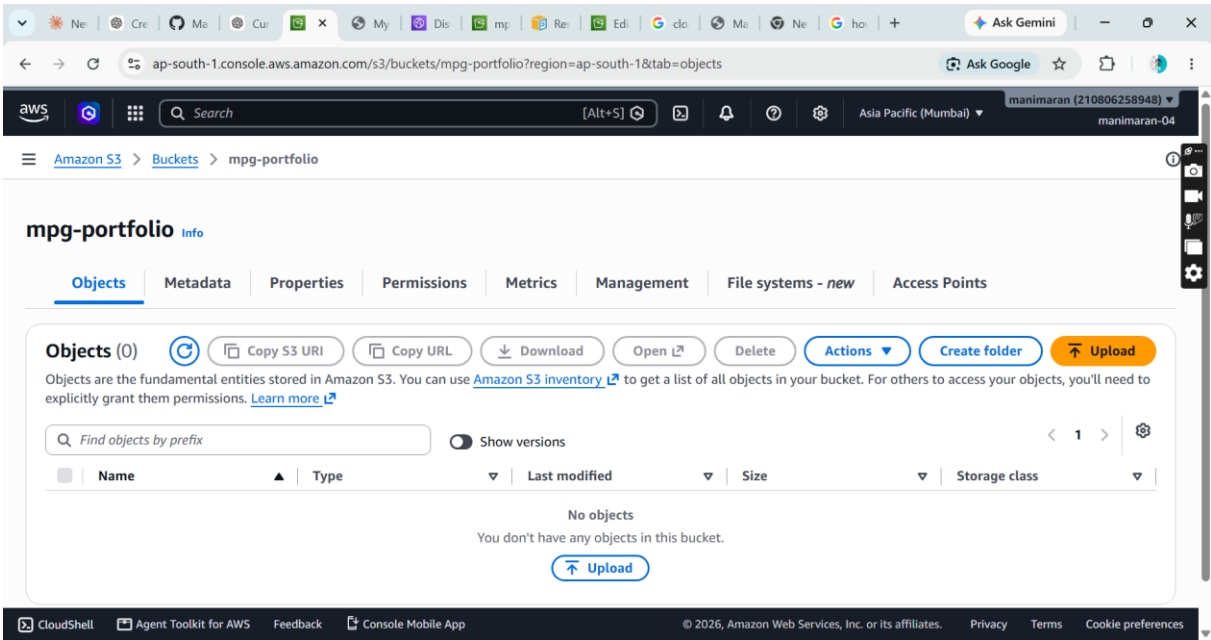


Figure 22. Verify the updated S3 bucket contents.

Step 22 — Check S3 object versions

Instruction / Action: The S3 object view with version information is inspected to verify the stored object history/version state.

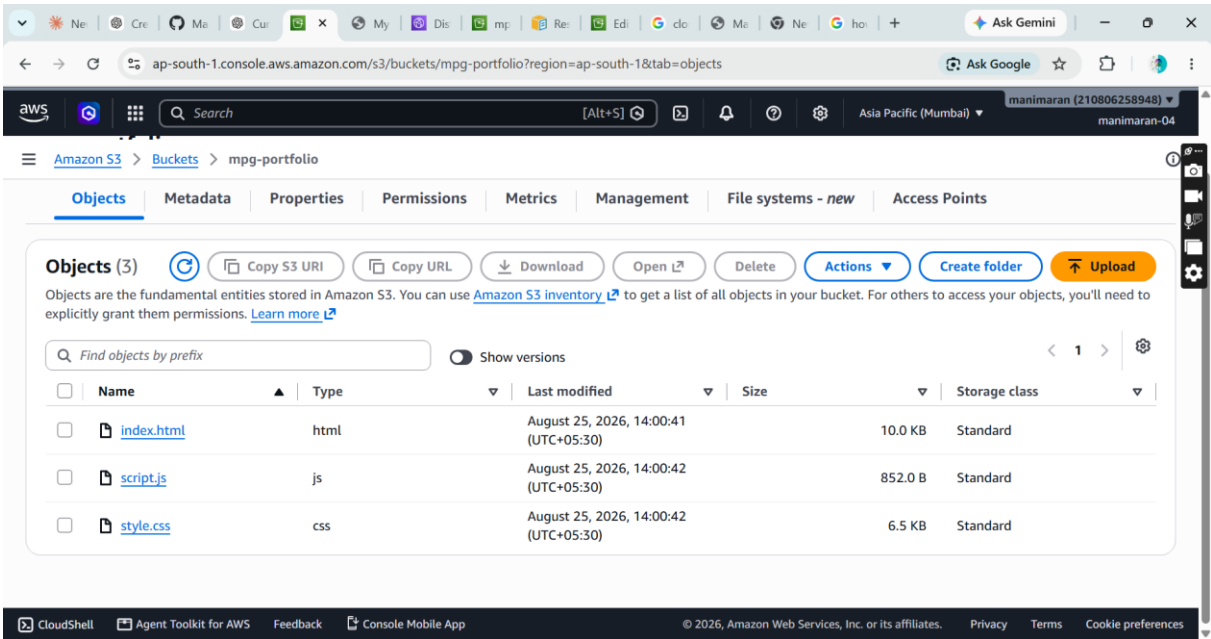


Figure 23. Check S3 object versions.

Step 23 — Final CloudFront validation

Instruction / Action: The portfolio website is rendered successfully through the CloudFront distribution after the update and validation steps.

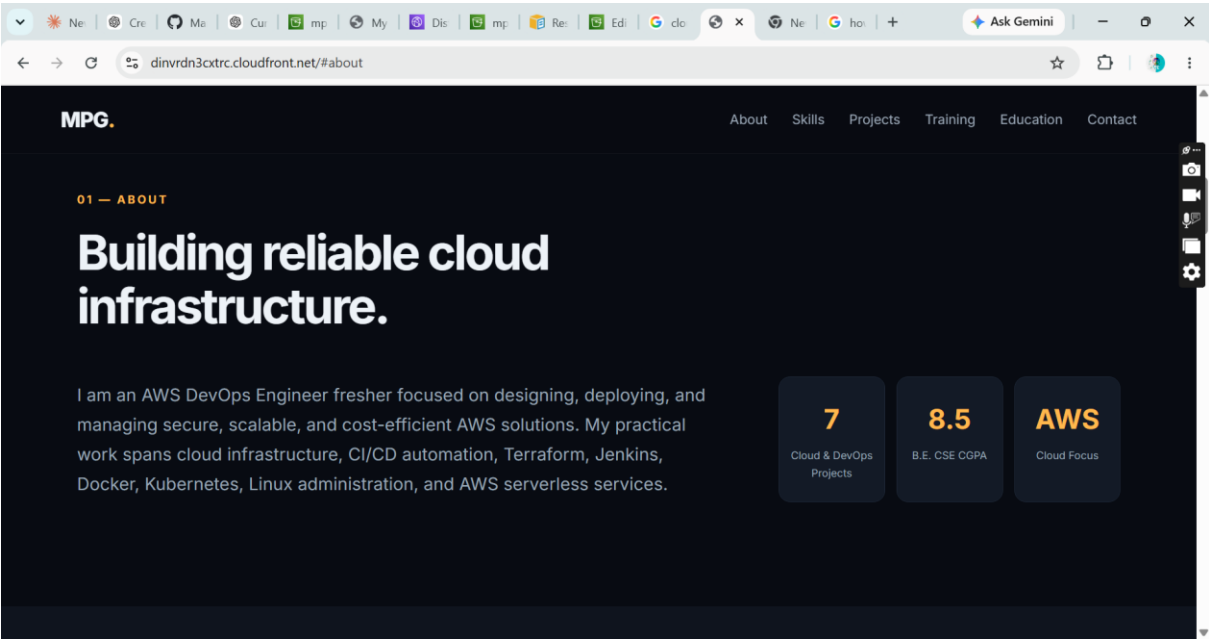


Figure 24. Final CloudFront validation.

10. Testing and Validation

Validation Area	Observed Evidence	Result
S3 bucket	Project bucket mpg-portfolio is visible in S3.	Passed
Initial website objects	Website files are visible in the S3 object listing.	Passed
CloudFront creation	A distribution is created and shown as enabled.	Passed
S3 origin	CloudFront origin points to mpg-portfolio.	Passed
Origin Access Control	OAC is configured/referenced for the S3 origin.	Passed
Bucket policy	CloudFront service-principal access policy is shown.	Passed
CloudFront domain	Distribution details expose a CloudFront domain.	Passed
Browser validation	Portfolio renders through the CloudFront domain.	Passed
AWS CLI	aws --version confirms CLI installation.	Passed
CLI S3 access	aws s3 ls displays S3 buckets including mpg-portfolio.	Passed
S3 update/version check	Updated objects and version information are visible.	Passed
Final CloudFront validation	Updated portfolio renders through CloudFront.	Passed

11. Troubleshooting and Operational Notes

- CloudFront may initially show connection/page-loading issues while the distribution or console content is still propagating or when the local network/firewall blocks required AWS console domains.
- When using CloudFront with a private S3 origin, the origin access configuration and bucket policy must correspond to the CloudFront distribution.
- When website files are changed in S3, CloudFront may continue serving cached content until the relevant cache expires or an invalidation is performed.
- The direct S3 object URL and the CloudFront URL represent different access paths; the intended public delivery path for this project is CloudFront.
- AWS CLI credentials must never be exposed in screenshots, documentation, repositories, or chat. Credential-bearing evidence has been redacted in this report.
- If the website does not update immediately, verify the S3 object first, then check CloudFront caching/invalidation.

12. Security Considerations

- Use Origin Access Control so CloudFront can access the S3 origin without requiring the bucket to be broadly public.
- Keep the S3 bucket policy limited to the CloudFront service principal and the intended distribution/resource.
- Use IAM users/roles with least-privilege permissions for AWS CLI operations.
- Do not store AWS access keys or secret access keys in source code, GitHub repositories, screenshots, or project reports.
- Rotate/revoke credentials if they are accidentally exposed and replace them with securely managed credentials.
- Use HTTPS through CloudFront for public website delivery.
- Review S3 Block Public Access and bucket policy settings carefully when configuring a CloudFront-backed website.

13. Advantages

- Serverless static website hosting without managing an EC2 web server.
- CloudFront provides a CDN layer for efficient delivery of static content.
- OAC supports a controlled CloudFront-to-S3 origin relationship.
- S3 provides durable object storage for website assets.
- AWS CLI enables repeatable deployment/update operations.
- The project establishes a foundation for later CI/CD and Infrastructure-as-Code automation.

14. Limitations

- The implementation is primarily a static website and does not provide server-side application processing.
- Content changes may require cache invalidation or waiting for cache expiry.
- Manual configuration of S3, CloudFront, OAC, and bucket policy can be error-prone.
- The evidence demonstrates deployment and validation but does not by itself constitute a full automated CI/CD pipeline.

15. Future Enhancements

- Automate S3 deployment with a CI/CD pipeline using GitHub and Jenkins.
- Automate CloudFront cache invalidation after each deployment.
- Manage S3, CloudFront, OAC, and related IAM/policies with Terraform.
- Add a custom domain name and managed TLS certificate.
- Add automated testing and deployment checks.
- Add CloudWatch monitoring and operational alerting where appropriate.
- Improve release/version management for website assets.

16. Key Lessons Learned

- Amazon S3 can be used as the storage origin for a static website.
- CloudFront separates content storage from global content delivery.
- Origin Access Control is important when the S3 origin should not be broadly public.
- CloudFront distribution configuration and the S3 bucket policy must work together.
- AWS CLI is useful for repeatable S3 inspection and deployment operations.
- Caching must be considered when updating static website files behind a CDN.
- Credentials shown in terminal screenshots must be protected and redacted in shared documentation.

17. Interview / Viva Explanation

“In my Level 1 AWS project, I deployed a static portfolio website using Amazon S3 and Amazon CloudFront. I created the mpg-portfolio S3 bucket and uploaded the HTML, CSS, and JavaScript files. I then created a CloudFront distribution and configured the S3 bucket as the origin. I used Origin Access Control and updated the S3 bucket policy so CloudFront could securely retrieve the website content. I validated the website through the CloudFront domain and HTTPS, and I also used AWS CLI from Windows to verify S3 access and perform website-file update operations. Finally, I revalidated the updated website through CloudFront and considered CloudFront caching during the update process.”

18. Conclusion

This Level 1 project demonstrates a complete static website delivery workflow using Amazon S3 as the storage origin and Amazon CloudFront as the content delivery layer. The evidence covers bucket creation and object management, CloudFront distribution creation, S3 origin configuration, Origin Access Control, bucket-policy configuration, browser validation, AWS CLI usage, content updates, and final CloudFront validation.

The demonstrated delivery path is: User Computer → Source Code → Amazon S3 → Amazon CloudFront → Public Website. This architecture provides the foundation for subsequent AWS DevOps work involving automation, CI/CD, Terraform, monitoring, containers, and Kubernetes.

Appendix A — Implementation Evidence Inventory

Figures	Count	Description
Figure 2 - 3	2 screenshots	Bucket list and mpg-portfolio object listing.
Figure 4 - 10	7 screenshots	Distribution creation, S3 origin, distribution status, origins, and OAC configuration.
Figure 11 - 12	2 screenshots	S3 bucket policy and CloudFront distribution details/domain.
Figure 13 - 17	5 screenshots	CloudFront website, local/source website, S3 object URL, and final browser checks.
Figure 18 - 24	7 screenshots	CLI version/configuration, S3 listing, local project directory, S3 objects and versions.
Figure 1	1 system architecture diagram	Computer → Source Code → Amazon S3 → Amazon CloudFront → Public Website.

END OF LEVEL 1 USER (WEB APPLICATION)→ S3 + CDN → INTERNET (WEB APPLICATION)